

Demo Runbook & Technical Validation Report

Post-quantum migration · crypto-agility · evidence-first scanning · project-specific policy drafts

Generated: 2026-09-26 16:17 India Standard Time

Validation result: 26 tests passed; seven demo validation steps recorded.

Important model disclosure: the current system does not contain a trained generative LLM. It includes a local PDF retrieval demo and a deterministic, evidence-based scanner. Unknown cost, performance, dependency, and runtime facts are intentionally not fabricated.

What was delivered

- The local web demo now has a public landing and demo-role selector, first-run organization setup, a combined cryptographic inventory/findings scanner, a PDF project-risk workflow, and a project-scoped firewall policy builder.
- The organization profile, scan history, and role selector are stored in browser local storage. Administrator and accessor are demonstration modes only; they are not server-authenticated identities, and the UI role is not an authorization boundary.
- The project PDF flow extracts scope, technology, timeline, budget, vendors, dependencies, and security requirements when the document contains evidence. It reports explicit unknown or insufficient-evidence states for absent data and offers JSON and print-to-PDF exports.
- The crypto scanner accepts PDF, ZIP, JSON/YAML, PEM/CRT, SBOM, and CBOM inputs. Findings include candidate algorithm matches, source file and line, redacted evidence where relevant, and a SHA-256 line hash. The report can be exported as JSON, CSV, or printed to PDF.
- The scanner includes synthetic sample project-PDF and software-ZIP inputs. A software archive report summarizes bounded static source files, observed languages, selected package manifests, dependency names, crypto candidates, and redacted secret markers. It does not run builds or claim dependency CVE coverage.
- The policy center builds a project-specific policy artifact, allows preview and draft save, and groups firewall events, CI/CD guard, and secure-data status in one place. A saved draft is not enforced by a live gateway and is not a production deployment.
- Synthetic repository, crypto asset, finding, and firewall event records are kept in a separate SQLite sample database and shown in the Sample Data view. They are not mixed into the organization scan history.
- Deployment issue intake accepts a reported issue and repository reference. It does not clone the repository or invent source/dependency correlations when those relationships have not been scanned.

Model and analysis behavior

- Important correction: this project has no fine-tuned or trained generative LLM. No model weights were trained, and no generative LLM API endpoint is configured for the risk-analysis or scanner workflows.
- The existing LocalCryptoLLM class is a local retrieval demo. It extracts PDF text with PyMuPDF, cleans the text, splits it into sentence-based chunks (250 characters with 40 characters of overlap), and embeds those chunks using the pretrained all-MiniLM-L6-v2 SentenceTransformer.
- The embedding vectors are normalized and indexed by FAISS IndexFlatIP. A question is embedded using the same pretrained encoder; the default top three similar reference chunks are retrieved and placed into a fixed response template. This is retrieval-augmented text assembly, not original generative reasoning or supervised training.
- The local demo was run against the provided `crypto_flaws_and_fixes.pdf` reference. It returned text about poor key management, weak padding, random number generation, and timing side channels. The run emitted a Hugging Face Hub unauthenticated-request warning while loading model weights; inference did complete. The source PDF was processed locally and was not sent as a prompt to a generative service.
- The upload scanner is deterministic pattern matching joined to the controlled recommendation data in `pqc_migrate/knowledge_repo.py`. It does not infer runtime reachability, validated financial cost, performance impact, complete certificate metadata, or dependency blast radius without dedicated evidence.
- The report now includes an initial USD engineering-effort estimate: default blended rate USD 150/hour, two baseline analysis hours, and editable effort assumptions of 8 hours per legacy-crypto finding, 16 hours per classical public-key migration finding, and 3 hours per other crypto review finding. The low/high range is 75%-150% of expected effort. This is a planning estimate, not an exact project or vendor cost; licenses, infrastructure, compatibility, validation, and deployment are excluded.
- Therefore the system does not claim that its advice is exhaustive, that estimates are measured, or that a system is quantum safe. Review candidate matches against current standards, implementation documentation, and organizational policy.

Upload safety and resource limits

- Uploaded repository content is read as data only. ZIP entries are never extracted to the filesystem and uploaded scripts are never run. Archive names are checked for absolute paths, drive prefixes, and parent traversal.
- The configured upload cap is 10 GiB per file. PDF analysis is limited to 500 pages and 2,000,000 extracted characters. ZIP analysis is bounded to 20,000 entries, 2 GiB expanded size, 512 MiB of scanned text, and 5 MiB per text entry.
- Possible private-key blocks and credential assignments are redacted from displayed evidence. A SHA-256 hash is supplied for source-line correlation; it is not a signature or proof that a finding is valid.
- The application stages uploads in a temporary file and removes the staged file after processing. Production deployment still needs reverse-proxy request limits, quotas and retention controls for the upload volume, concurrency limits, background workers, and isolated scan containers.
- The 10 GiB application setting alone does not guarantee that a 10 GiB request can pass through a browser, proxy, hosting platform, or operating system. Those infrastructure limits must be configured and tested independently.

Seven-step validation

1. Static source checks - `node --check frontend/app.js` exited successfully. Pylance reported no Python syntax errors in `backend/app/main.py` or `src/crypto_llm.py`, and no diagnostics remained in those checked Python files.
2. Dependency consistency - `.venv Python -m pip check` completed with: No broken requirements found.
3. Automated tests - the full pytest suite completed with 26 passed. Non-fatal Starlette, Pydantic, and FastAPI deprecation warnings remain in the existing application/tests.
4. Local reference retrieval - the original PDF was passed to `src/crypto_llm.py`. The `all-MiniLM-L6-v2` encoder loaded, the FAISS retrieval returned relevant PDF passages, and the command completed. This validates retrieval execution, not model training or answer completeness.
5. Project risk PDF flow - the original PDF was uploaded in the browser and produced a `REVIEW_REQUIRED` report. Presently absent scope, budget, vendors, dependency, and security fields remained unknown or insufficient evidence; cost was not fabricated; the report disclosed that no generative LLM was configured.
6. Cryptographic PDF scan - the original PDF was scanned in the browser. The saved scan summary contained 79 candidate evidence items across 16 observed labels: 3DES, AES, DES, DH, ECC, HMAC, MD5, ML-DSA, ML-KEM, RC4, RSA, SHA-1, SHA-2, SHA-3, SLH-DSA, and TLS. Candidate matches still require validation.
7. Integrated UI and API - the landing page and both demo-role selections were exercised. The accessor Settings screen had disabled profile controls and no save action. All 18 workspace navigation views rendered with zero captured page errors. The synthetic project PDF returned a risk report and editable-assumption USD estimate; the synthetic software ZIP returned source, dependency, crypto, and cost analysis without execution. Browser CSV export was verified to include the estimated cost, dependencies, and crypto evidence while omitting the redacted placeholder secret. A project policy draft remains undeployed.

Current boundaries and production work

- Persistence: organization setup, scan summaries, and policy drafts are not persisted to the product PostgreSQL database. Organization preferences live in this browser; firewall draft storage is local/in-memory demo state and can disappear after a restart.
- Identity and access: demo role selection is client-side. Add OIDC/JWT, MFA, server-side RBAC, tenant isolation, CSRF protections where applicable, and audited authorization before protecting real customer data.
- Integrations: GitHub/GitLab OAuth and cloning, real deployment telemetry, background job dispatch, remediation containers, KMS/HSM envelope encryption, a generative LLM, and an Envoy/OPA enforcement gateway are not configured.
- Analysis completeness: text matching is not a full language-semantic scanner. Certificate parsing, dynamic algorithm flow, wrapper resolution, transitive package graphs, service blast radius, and compatibility/performance tests require dedicated integrations and real repository evidence.
- Automation: selected report cadence is currently a saved preference, not a scheduled report job. The policy output is a reviewable draft, not an automatically deployed firewall. High-impact changes must remain approval-gated.
- Before production, connect persistent storage and job infrastructure; enforce proxy and worker resource limits; integrate identity, KMS/HSM, repository providers, vetted scanning and sandbox images, policy gateways, retention controls, monitoring, and disaster recovery; then repeat the adversarial suite in the target deployment environment.

Run the local demo

- From the repository root, start the app with: `.venv\Scripts\python.exe -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000`
- Open `http://127.0.0.1:8000/`. Select Administrator for the organization setup and policy draft demo, or Accessor for the read-only profile view. Do not use these local roles to protect real data.
- Run tests with: `.venv\Scripts\python.exe -m pytest -q`
- Check frontend syntax with: `node --check frontend\app.js`
- Regenerate this report with: `.venv\Scripts\python.exe scripts\generate_demo_report.py`
- The report records the observed state of this local prototype. It is not a compliance attestation, a penetration-test report, or a production-readiness certification.